Contents

Numera Voice Module — Audit & Fix Plan

Date: 2026-07-21 **Trigger:** Session review — mid-conversation sentence dropped, no barge-in, transcription feels split, response attribution confusing in long conversations, and the whole thing feels disjointed ("modules feel separate, not like a tutor"). **Scope:** Diagnoses each reported issue against the actual code, then splits the fix into **Frontend** (this repo) and **Backend** (Aditya / voice server `:8004`) work. The Backend section is written so it can be handed off directly.

TL;DR — the five issues

#	Symptom (from the session)	Primary cause	Owner
1	Mid-conversation a whole sentence was dropped / "just skipped"	Mic stays live during TTS (no half-duplex) + Web Speech self-restart gap (rest) / Deepgram endpointing fired mid-utterance (server)	**Backend-led**, Frontend contributes
2	No barge-in — can't talk over the tutor like OpenAI/Gemini	Nothing stops TTS when the student starts speaking; no interrupt path	**Frontend + Backend**
3	"Transcribing…" then the real text appears elsewhere; wants ONE text box	Live caption and final text live in two different stores / two surfaces	**Frontend**
4	In long convos, unclear which student text a reply answers	No turn id, no sequencing, async turns can race and interleave	**Frontend + Backend**
5	Feels disjoint — modules aren't integrated	No single "conversation turn" state; each surface updates on its own channel	**Frontend (architecture) + Backend contract**

How the pipeline works today (ground truth)

There are **two transports**, chosen by `NEXT_PUBLIC_VOICE_TRANSPORT` (`app/page.tsx:29`):

- **`rest` (current default):** browser does STT + turn detection. `useVoiceTurn` runs a VAD (Web Audio energy + silence timer) to decide turn-end and the Web Speech API for the transcript, then fires `onTurnEnd` → `submitVoiceTurn` (REST `/interaction`) → `speakTutor()` voices the reply (backend `/voice/tts`, browser fallback).
- **`server`:** `useVoiceStream` streams raw PCM to `:8004`; the voice server does Deepgram STT + tutor + streamed MP3 TTS, pushed back over the `/voice` WebSocket (`useWebSocket` → `tutorAudioStream`).

Key state lives in two stores: `useMicLevel` (live `caption`, `aiSpeaking`, mic levels) and `useNumeraStore` (`transcript[]`, phase, canvas).

Issue 1 — Mid-conversation dropout / a whole sentence skipped

What the code does: - Mic capture is started/stopped **only** on `micMuted` (`app/page.tsx:119-123`). It is **not** paused while the tutor is speaking. So during TTS playback the mic is still open and the recognizer is still running. - `rest` path: Web Speech `continuous = true` restarts itself on `onend` (`useVoiceTurn.ts:186-195`). On each restart the browser discards un-finalized interim text and leaves a short deaf gap. If the silence timer fires `commitTurn` during that gap, the trailing phrase is lost. The `caption`-vs-`transcriptRef` "take the longer" fallback (`useVoiceTurn.ts:105-107`) softens this but can't recover a full restart gap. - `server` path: a cleanly dropped sentence points at Deepgram **endpointing firing mid-utterance** (UtteranceEnd/`speech_final` too aggressive), or a socket keepalive/reconnect that drops `audio_chunk` frames.

Most likely primary cause of "it didn't catch my answer, just skipped": the tutor's own audio is playing through the speaker while the mic is open, so either (a) the recognizer treats the tutor's voice as the student and overwrites the turn, or (b) the backend endpointer cuts the student off. This is the **half-duplex** problem — it underlies both Issue 1 and Issue 2.

Fix — Frontend: 1. **Half-duplex gating (rest):** while `aiSpeaking` is true, either pause the recognizer or, better, wire barge-in (Issue 2) so student speech instantly stops the tutor and re-opens a clean turn. 2. Restart Web Speech **eagerly on a timer** (before the browser's own `onend`) so there's no deaf gap, and keep the interim buffer across restarts.

Fix — Backend (see hand-off): tune Deepgram endpointing, add keepalive + gap-free reconnect, and never transcribe the tutor's own TTS.

Issue 2 — Barge-in (the OpenAI / Gemini behaviour)

Requested behaviour (Manjusha): when the user starts talking, the LLM **stops the TTS immediately**; the already-spoken/queued content is put in a buffer; when the user stops, the new STT is combined with that buffer and processed.

What the code does: nothing. `useVoiceTurn` never reads `aiSpeaking` and never calls `stopTutorSpeech()` / `tutorAudioStream.stop()`. The tutor talks over the student to the end of the clip. The stop primitives already exist (`lib/tts.ts:54 stopTutorSpeech`, `tts.ts:194 TutorAudioStream.stop`) — they're just never triggered by speech onset.

Fix — Frontend (rest path, doable now): - In `useVoiceTurn`'s VAD, on **speech onset** (`rms > energyThreshold` transitions false→true) **while `useMicLevel.getState().aiSpeaking` is true**: - call

`stopTutorSpeech()` and `tutorAudioStream.stop()` immediately; - mark the interrupted tutor message (e.g. append " …" / a subtle "interrupted" style) so the transcript stays honest; - keep the buffer: the words the tutor had *already said* are what the student is reacting to — retain the current tutor message text as context for the next turn. - Guard against **self-trigger**: because the mic hears the speaker, only treat onset as barge-in when energy clears a higher threshold than the TTS bleed-through, and rely on `echoCancellation` (already on). Half-duplex gating from Issue 1 makes this robust.

Fix — Backend (server path): a client→server `{ type: "interrupt" }` control that halts TTS generation server-side and flushes the queued/undelivered audio, plus don't feed the tutor's own audio back into STT. Frontend already has `sendControl` (`useWebSocket.ts:174`) to send it.

Issue 3 — One text box for transcription (not "transcribing…" then a separate bubble)

What the code does: the live interim caption lives in `useMicLevel.caption` and is rendered in `VoiceBar`. The **final** text is a *separate* new bubble appended to `useNumeraStore.transcript` (`Transcript.tsx`). In the rest path the caption is **cleared** on commit (`useVoiceTurn.ts:111`) and a brand-new student bubble is added inside `submitVoiceTurn` (`useDemoTutor.ts:246`). In the server path a dashed "partial" bubble (`updatePartialTranscript`) is replaced by the final. Either way the text visibly **jumps between two places**, which is the "I get confused which text it's for" feeling at the input stage.

Fix — Frontend: - Render the **live interim as an in-place evolving student bubble in the same `Transcript` list** — promote `caption` into a single `partial` student message that mutates as words arrive, then flips to `partial: false` on commit (reuse the existing `updatePartialTranscript` shape for the rest path too). - Remove the duplicate: don't clear the caption and re-add a fresh bubble; convert the same bubble. One text box, partial → final, no jump.

Issue 4 — "Which text is the reply answering?" in long conversations

What the code does: - Turns are **fire-and-forget**: `onTurnEnd → void submitVoiceTurn(...)` (`app/page.tsx:76-90`). Each call is an independent async REST request. - There is **no turn id and no sequencing**. If the student pauses > `silenceMs` (1300ms) mid-thought, **two turns fire**, two `/interaction` calls race, and replies can append **out of order**. Nothing cancels a superseded turn or binds a reply to the turn that caused it. - Combined with no barge-in (Issue 2), reply N can still be *playing* when turn N+1 fires — so the reply to N+1 reads as if it answered N.

Fix — Frontend: - Assign each turn a monotonically increasing `turnId`. Track the latest; **ignore/cancel responses for stale turns** (same `speakToken` pattern already used in `tts.ts`). - While a reply is in flight, either **queue** the next turn or hold commit until the reply lands, so turns are strictly serialized. - Optionally render student-turn → tutor-reply as a **visually linked pair** so attribution is obvious.

Fix — Backend: echo a `turn_id` / `in_reply_to` on every `tutor_response` and `/interaction` response so the client binds reply→turn deterministically instead of guessing by arrival order.

Issue 5 — "Feels disjoint, modules aren't integrated"

Why it feels separate: there is no single conversation-turn concept. The transport is split in two (`rest` vs `server`), the caption lives in one store and the transcript in another, TTS runs in two engines, and canvas / visual-cue / phase each update on their own channel at their own time. The surfaces are technically correct

but **never share one source of truth for "where are we in this turn,"** so they animate independently and the session doesn't feel like one tutor.

Fix — Frontend (the integration work): - Introduce a single **turn state machine**: `idle → listening → thinking → speaking → interrupted`, in one store, that every surface subscribes to (avatar mouth, mic bar, transcript bubble, canvas, visual cue, mic button). - Model **one turn object** carrying everything for that exchange: `{ turnId, studentText, tutorText, audio, canvas_draw, phase, visualCue, status }`. Surfaces read from the turn, not from five separate signals. - Unify the two transports behind one interface so the UI code is identical whether STT/TTS is browser or server — only the adapter differs.

This is the piece that makes it feel like a tutor rather than assembled parts. It also naturally absorbs Issues 2–4 (barge-in flips state to `interrupted`; the turn object gives attribution; the single caption bubble is just the turn's `studentText` while `status = listening`).

Backend hand-off — requirements for the voice server (`:8004`) / Aditya

These are the items the frontend **cannot** fix alone. Each maps to a symptom above.

1. **Barge-in / interrupt (Issue 2, 1).**
   - Accept a client control message `{ "type": "interrupt", "turn_id": <n> }`.
   - On receipt: **stop TTS generation immediately** and stop sending `tutor_audio_chunk`; send `{ "type": "tutor_audio_end", "total_chunks": <delivered>, "interrupted": true }`.
   - Do **not** feed the tutor's own TTS audio back into STT (server-side echo/self-capture guard). This is the single biggest cause of dropped/garbled turns.
2. **Robust endpointing (Issue 1).**
   - Deepgram: use `interim_results` + `utterance_end_ms` tuned so a **natural mid-sentence pause does not** trigger `UtteranceEnd`. Prefer `speech_final` + a configurable `utterance_end_ms` (~1000–1500ms) over aggressive endpointing.
   - Emit `transcript_partial` continuously and a single `transcript_final` per real utterance; never split one spoken sentence into two turns.
3. **Gap-free audio ingestion (Issue 1).**
   - Keepalive the Deepgram socket; on reconnect, **buffer incoming `audio_chunks`** so no frames are dropped during the reconnect. A silent reconnect currently reads as "it skipped my whole answer."
4. **Turn correlation (Issue 4).**
   - Include `turn_id` (and `in_reply_to`) on every `tutor_response`, `transcript_final`, and `/interaction` REST response, echoing the client's turn id. Lets the client bind each reply to the exact student turn.
5. **Combine-on-resume semantics (Issue 2).**
   - After an interrupt, when the next `transcript_final` arrives, process it **with the interrupted turn's context** (the student is usually continuing/correcting), per the OpenAI/Gemini behaviour Manjusha described: buffer the prior content, combine with new STT, then respond.
6. **(Nice to have) Streamed reply text token-by-token** so the on-screen tutor text can stream in sync with the audio, reinforcing the "one tutor" feel (Issue 5).

Wire messages the frontend already supports: `sendControl(type, extra)` exists (`useWebSocket.ts:174`) to send `interrupt`; inbound `tutor_response` / `tutor_audio_chunk` / `tutor_audio_end` are already handled — only the new fields (`turn_id`, `interrupted`) need adding on both sides.

Suggested sequencing

Frontend, now (no backend dependency): 1. Barge-in in `useVoiceTurn` (stop TTS on speech onset while `aiSpeaking`). — Issue 2 2. Half-duplex / eager recognizer restart. — Issue 1 (rest) 3. Single evolving transcript bubble (partial → final in one place). — Issue 3 4. Turn ids + serialize/guard stale responses. — Issue 4

Frontend, next: 5. Unified turn state machine + one turn object; unify the two transports behind one adapter. — Issue 5

Backend, in parallel (hand-off section): items 1–5, with barge-in + endpointing + no-self-transcribe first (they fix the worst symptom).

Verification per fix: reproduce the exact session symptom (talk over the tutor; pause mid-sentence; speak slowly) and confirm — TTS stops on barge-in; no sentence dropped; one text box; each reply pinned to its turn.